# LightOpt：面向实时渲染的光源优化

## LightOpt: Lights Optimization for Real-time Rendering

Tuo Chen*（清华大学，中国） · Luyan Cao†（LIGHTSPEED，中国）
Kui Wu（LIGHTSPEED，美国） · Shimin Hu（清华大学，中国）

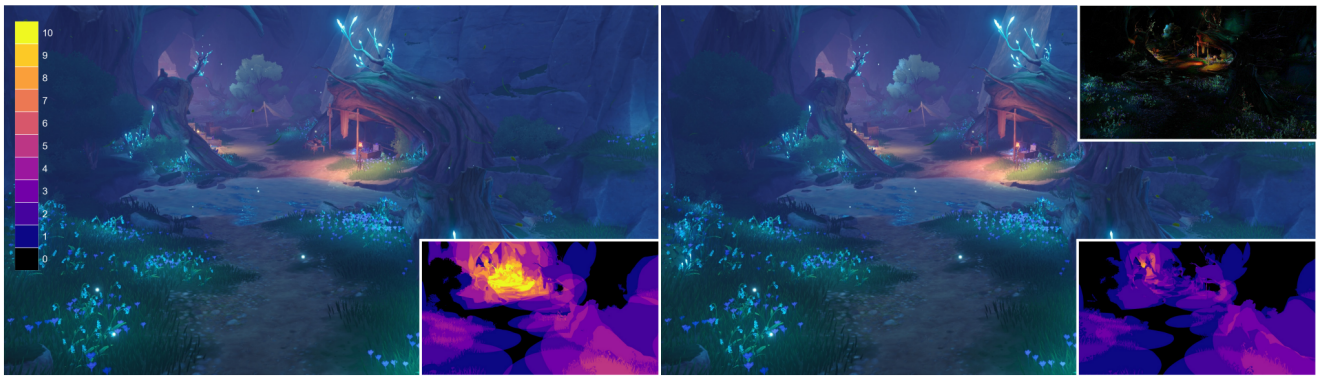

图 1 输入游戏场景有 121 盏灯（左）。我们在可微渲染流程中引入新的损失函数，自动减少光源并保留原貌（右）。优化后剩下 57 盏灯，场景中不同位置的加速比为 1.3 至 2.1 倍。右上角为相对原图逐像素差异放大 5 倍后的结果（RMSE = 0.0552）。右下角为光照重叠热图，黄色表示高、黑色表示低。该视角的平均光照重叠数（ALO）从 2.4 降至 1.7。

## 摘要

现代游戏场景通常需要数百盏灯才能呈现丰富的视觉细节。然而，为每个像素计算多盏灯的贡献仍是实时渲染的主要性能瓶颈，在低端平台上尤其如此。我们提出一种可微光照优化流程，在保持最终渲染外观的同时自动减少活跃光源。该方法联合优化光源参数，并在优化过程中动态删除、合并和添加光源。我们引入两个损失函数，以减少冗余的光照重叠并防止局部照明不足，还将方法扩展到昼夜时变照明。在实际游戏制作场景中，光照开销下降了 18% 至 52%；在视觉保真度方面，该方法始终优于美术人员手工精简灯光的结果。

ACM 计算分类 计算方法 → 渲染。

ACM 引用格式 Tuo Chen、Luyan Cao、Kui Wu、Shimin Hu，2026。《LightOpt：面向实时渲染的光源优化》。SIGGRAPH 2026 会议论文集，2026 年 7 月 19–23 日，美国加利福尼亚州洛杉矶。ACM，美国纽约，原文 10 页。DOI 10.1145/3799902.3811072。ISBN 979-8-4007-2554-8/2026/07。

* 本工作完成于 Tuo Chen 在 LIGHTSPEED 实习期间。† 通讯作者。

© 2026 原权利人／作者。原作采用知识共享署名 4.0 国际许可（CC BY 4.0）。本版为非官方中文翻译，正文、图注及表格已翻译并重新排版，插图沿用原作。原作与许可链接见页末。

作者原版 PDF
CC BY 4.0 国际许可

# 1 引言

照明决定了游戏场景中哪些内容易于辨认、空间深度如何呈现，以及玩家会产生怎样的情绪反应。为了忠实表现复杂环境，现代游戏往往使用大量光源。设计师借助多盏灯营造氛围、突出兴趣点，并让场景各区域的照明随游戏事件动态变化。因此，单个场景包含数百盏灯并不罕见，在视觉元素密集的区域，可能有 10 至 50 盏灯重叠。实时渲染的光照开销随每个可见像素需要计算的灯数增长。双向反射分布函数（BRDF）的求值、光源影响随空间变化导致的像素执行分歧，以及缓存局部性下降，都使新增光源带来可观开销，低端 GPU 尤为明显。尽管已有分块光源剔除［Andersson 2009；Lauritzen 2010；Olsson 与 Assarsson 2011］、分簇光源剔除［Olsson 等 2012］和 Tiled-Z 剔除［Drobot 2017］等技术，高效渲染含大量光源的场景依然是个难题。

跨平台开发进一步放大了这个问题。玩家希望 PC、主机和移动设备能提供相近的照明质量。受计算预算限制，低端平台要维持交互帧率，只能启用少量光源。为高端平台维护一套多灯配置，同时为低端平台维护另一套灯数更少、外观相近的配置，既耗费人力，也难以长期维护，给灯光美术和制作流程带来负担。

本文提出首个在减少活跃光源时联合优化位置、强度、颜色和方向等参数，以保持场景最终渲染外观的可微光照优化流程。框架允许在优化过程中动态删除、合并和添加光源。为引导这一过程，我们提出两个新的损失函数，分别抑制冗余重叠和防止照明不足。我们还支持昼夜时变光源（Time-of-Day，ToD），这是游戏用来模拟一天中自然光照变化的常见功能。多个真实游戏场景的实验表明，活跃光源减少 20% 至 53%，总体光照开销降低 18% 至 52%。相较于美术人员手工精简，自动优化只需数分钟。与业界常用的视角相关方法相比，我们在延迟着色耗时和最终渲染图像的均方根误差（RMSE）上均取得更好的结果。跨平台性能评测也验证了这种方法的有效性。

# 2 相关工作

## 多光源问题

与离线渲染相比，实时渲染的计算预算十分有限。仅计算多光源直接照明就很有挑战，能够处理的光源数量通常限于数百到数千［Dachsbacher 等 2014］。实时光栅化最常用的处理方式是光源剔除：把视锥划分为多个子视锥，记录影响各子视锥的光源并维护相应列表，着色时让每个像素使用所属子视锥的列表。分块剔除把屏幕划分为规则的二维图块，每个图块对应一个子视锥［Andersson 2009；Lauritzen 2010；Olsson 与 Assarsson 2011］。分簇剔除加入深度轴，把二维划分扩展为三维网格，给每个簇（子视锥）单独建表，从而更精确地剔除光源［Olsson 等 2012；Persson 2013］。Tiled-Z 剔除引入深度分箱（Z-bins），既保留二维图块的简洁性，又能在很大程度上区分深度［Drobot 2017］。

在光线追踪管线中，主要开销来自光源采样。光源层次结构允许对大量光源进行重要性采样，但构建和维护这类结构的代价不可忽略［Conty Estevez 与 Kulla 2018；Lin 与 Yuksel 2020；Moreau 等 2022；Yuksel 2019］。自适应直接光照概率密度函数和路径引导方法［Dahm 与 Keller 2017；Fan 等 2023、2025；Herholz 等 2016；Hey 与 Purgathofer 2002；Jensen 1995；Lafortune 与 Willems 1995；Li 等 2022；Liang 等 2024；Müller 等 2017；Vorba 等 2019、2014］通常需要对每个像素做大量采样。较新的方法借助额外的数据结构或简化的分布达到了实时性能［Dittebrandt 等 2023；Lu 等 2024；Zeng 等 2025］。ReSTIR［Bitterli 等 2020］用重采样重要性采样提高效率，但也引入了较大的常数开销［Wyman 与 Panteleev 2022］。在实践中，虚幻引擎 MegaLights［Narkowicz 与 Costa 2025］等系统利用追踪得到的可见性信息，为每个图块建立可见光源列表，再将其作为随机直接照明的采样先验。总体而言，这些方法从算法层面改进采样或剔除，保留原有的渲染光源集合。另一类工作关注复杂灯具的表示和采样［Atanasov 与 Koylazov 2025；Velázquez-Armendáriz 等 2015；Zhu 等 2021］。本文研究的是场景中多个简单光源的整体效果。

### LightCuts

LightCuts 及其变体与本文有相近的总体目标，都是用紧凑的光源集合近似原始照明，在保留外观的同时降低多光源渲染开销。Walter 等［2006、2005、2012］为原始光源建立层次表示，并为每个像素选出一组光源（lightcut），用以近似全部光源的贡献。MRCS［Hašan 等 2007、2008］计算所有像素共享的统一切分，LightSlice［Ou 与 Pellacini 2011］则把这一思路扩展到按图块切分。这些方法需要针对当前视角评估切分，实时渲染中每当视口变化，重新计算的代价都很高。其基本思想是在运行时从固定光源集合里挑选具有代表性的子集，从而利用光源之间的冗余。我们的框架直接简化光源集合：通过优化和重组全场景的照明配置，生成一套新的紧凑光源集合。它可与运行时光源选择算法配合，作为预处理同时减少光源总数和各子视锥的光源列表长度。

## 可微渲染的应用

可微渲染已广泛用于逆向图形学，包括几何重建、物理模拟、材质与纹理估计以及光照优化，例如通过优化方差来调整场景参数［Yan 等 2024］。许多方法利用可微渲染［Li 等 2018；Nimier-David 等 2019］从二维图像恢复三维几何、照明和材质属性［Hasselgren 等 2022；Ling 与 Zhang 2023；Munkberg 等 2022；Xie 等 2024］，或用于简化网格［Hasselgren 等 2021］、优化发片几何［Zheng 等 2025］、联合优化 UV 和纹理［Knodt 等 2023］。这些方法通常以屏幕空间的重建指标为目标。我们建立多目标优化问题，直接考虑在场景几何表面采样得到的辐照度。Lipp 等［2024］使用伴随光线追踪连续优化照明配置，实现目标驱动的灯光设计。本文关注实时渲染，并在优化时通过删除、合并和添加光源，离散地重组光源集合。

# 3 可微光源优化

本节先给出问题定义和方法概览，然后详细介绍各个组成部分。

## 3.1 问题定义与方法概览

实时渲染中，光照开销随每个可见像素需计算的光源数增长。每增加一盏灯，都需要额外计算 BRDF。由于光源影响具有空间差异，像素执行分歧也会增加，缓存局部性则会下降。这些影响在低端 GPU 上尤为明显。因此，我们在渲染管线下求解一个多目标优化问题，在视觉保真度与光源复杂度之间折中。

## 辐照度场

理想情况下，我们希望减少光源，同时保留原有视觉效果。但渲染方程中的 BRDF 位于半球积分内部，难以直接优化。我们用辐照度场作为目标，近似描述外观。场景表面一点的辐照度是该点单位面积接收的总辐射功率，通过对表面上方半球内所有入射方向积分得到。设 x 为单位法线为 n 的表面上的一点。光源集合 L 在该点产生的辐照度 E(x) 为

$$E(\mathbf{x}) = \sum_{l_i \in \mathcal{L}} \int_{\Omega^+} L_i(\mathbf{x}, \boldsymbol{\omega})\ \mathbf{n} \cdot \boldsymbol{\omega}\ d\boldsymbol{\omega}\ = \sum_{l_i \in \mathcal{L}} E_i(\mathbf{x})\ , \qquad (1)$$

其中，$\Omega^+$ 为以 n 为轴的半球，$L_i$ 和 $E_i$ 分别表示第 i 个光源产生的入射辐亮度和辐照度。

## 可微光源优化框架

给定一个含有多个光源的游戏场景，我们先对场景表面均匀采样，计算原始辐照度场，并将其作为优化目标。算法迭代优化原始光源，每轮交替执行两个子步骤。第一步固定光源数量，求解下式来调整光源参数，以保留原辐照度场。

$$\underset{\{\mathbf{c}_i, I_i, \mathbf{s}_i, \mathbf{d}_i, \dots\}}{\arg\min} \int_{\mathrm{S}} (\hat{E}(\mathbf{x}) - E(\mathbf{x}))^2 ds + a^{\mathrm{o}} l^{\mathrm{o}} + a^{\mathrm{a}} l^{\mathrm{a}}\ , \qquad (2)$$

S 表示场景的可见表面，花括号中为光源配置参数（详见下文的光源模型说明）。E(x) 和 Ê(x) 分别表示原始和优化后的表面辐照度。a 和 l 上的对应上标区分正则项的权重和损失，后文将分别说明。我们用辐照度驱动的可微渲染框架求解此问题。第二步更新光源数目，删除不重要的光源、合并冗余光源，并为照明不足的区域添加光源。算法 1 给出整个流程。

## 算法 1 优化流程

1 构建采样集合 S_all（第 3.2 节）。
2 计算每个采样点的原始辐照度 E(x)。
3 对每次迭代，执行以下步骤：
4 利用可微渲染最小化式（2），直至收敛（第 3.3、3.4 节）。
5 删除不重要的光源（第 3.5 节）。
6 合并光源（第 3.6 节）。
7 添加新光源（第 3.7 节）。
8 结束循环。

## 光源模型说明

框架本身并不限于特定光源类型。本文主要考虑游戏业最常用的点光源和聚光灯。各引擎对聚光灯的实现有所不同。我们采用 Unity 的光照模型［Unity 2023］，该模型最早见于 Frostbite［Lagarde 与 De Rousiers 2014］，现已广泛用于实时游戏。第 i 个光源在点 x 产生的入射辐照度建模为

$$E_i(\mathbf{x}) = V_i(\mathbf{x})\ \mathbf{c}_i I_i D_i(\mathbf{x})\ A_i(\boldsymbol{\omega}_i)(\mathbf{n}\cdot\boldsymbol{\omega}_i)\ , \tag{3}$$

$V_i(x)$ 是可见性项，本文假设其为 1。$\omega_i$ 是从 x 指向光源位置 $s_i$ 的单位方向。$c_i$ 和 $I_i$ 分别表示颜色和强度。$D_i$ 和 $A_i$ 分别表示距离衰减和角度衰减，定义如下。

$$D_i(\mathbf{x}) = \frac{a_i}{a_i + ||\mathbf{s}_i - \mathbf{x}||^2}\max(0.0, 1.0 - \frac{||\mathbf{s}_i - \mathbf{x}||^2}{r_i^2})^2\ , \tag{4}$$

$$A_i(\boldsymbol{\omega}) = \begin{cases} \text{clamp}(\frac{(-\boldsymbol{\omega})\cdot\mathbf{d}_i - \cos\theta_i^o}{\cos\theta_i^i - \cos\theta_i^o}, 0.0, 1.0)^2, & \text{if spotlight} \\ 1.0, & \text{otherwise} \end{cases} \tag{5}$$

式（5）第一行适用于聚光灯，第二行适用于其他情况。$a_i$ 是衰减平滑参数，$r_i$ 是光源作用半径，$d_i$ 为聚光灯方向。带上标 o 和 i 的 $\theta_i$ 分别表示外锥角和内锥角。对于点光源，角度衰减 $A_i(\omega)$ 恒等于 1。

## 3.2 离散化

为高效计算式（1）的辐照度场及其他正则项，我们将场景表面离散化为采样点，如图 2 所示。先用泊松盘采样［Yuksel 2015］在场景表面均匀取点，再把每盏灯的作用半径扩大到原来的两倍，将能覆盖到的所有采样点记为 S_all。其中既有原本照亮的点，也有原本未照亮的点。优化有两个目标：准确保留亮处的辐照度，并让暗处继续保持黑暗。常规游戏开发通常预烘焙多次反弹产生的全局照明，因此本文只考虑直接照明。式（2）的第一项可改写为

$$\int_S (\hat{E}(\mathbf{x}) - E(\mathbf{x}))^2 ds \approx \Delta s \sum_{\mathbf{x}\in\mathcal{S}_{\text{all}}} \left(\hat{E}(\mathbf{x}) - E(\mathbf{x})\right)^2\ , \tag{6}$$

$\hat{E}(x)$ 为优化后光源集合中各光源对该点辐照度之和。我们只在 S_all 中求值，$\Delta s = \pi r_0^2$ 表示每个采样点代表的面积，$r_0$ 是泊松盘采样半径。

图 2 场景表面上的采样点示例。

## 3.3 重叠损失

在分块渲染［Olsson 与 Assarsson 2011］、Tiled-Z 渲染［Drobot 2017］和分簇渲染［Olsson 等 2012］等实时管线中，每个图块的计算开销大致正比于块内像素数与相应光源列表长度的乘积。逐视角、逐图块地直接优化这些列表并不现实。为此，我们引入近似重叠损失，对表面采样点处的过度光照重叠施加惩罚。实践中，这能缩短图块光源列表的平均长度，提高渲染效率。

在每次迭代中，先找出能够影响表面采样点 x 的光源集合 $L_x$，再对所有采样点定义全局重叠惩罚：

$$l^{\mathrm{o}} = \sum_{\mathbf{x}\in\mathcal{S}_{\mathrm{all}}} \left[\alpha(|\mathcal{L}_{\mathbf{x}}|) \sum_{l_i\in\mathcal{L}_{\mathbf{x}}} w_i(\mathbf{x})\, d_i(\mathbf{x})\right], \tag{7}$$

$|L_x|$ 表示该集合的大小。缩放函数定义为

$$\alpha(n) = (\max(0, n - n^{\max}))^2 \tag{8}$$

当影响一个采样点的灯数超过用户指定的最大允许重叠阈值 n_max，函数就会施加惩罚，从而在维持足够照明的同时，使光源覆盖更稀疏。

为了优先移除贡献较弱的灯、保留起主导作用的灯，我们定义软权重函数

$$w_i(\mathbf{x}) = \frac{\exp\left(-\beta_w\, E_i(\mathbf{x})\right)}{\sum_{l_j\in\mathcal{L}_{\mathbf{x}}} \exp\left(-\beta_w\, E_j(\mathbf{x})\right)}, \tag{9}$$

ß_w 控制权重的陡峭程度。权重与第 i 盏灯的辐照度 $E_i(x)$ 反向变化。评估光源的影响时使用两倍作用半径，如图 3 所示，从而在光源移动并跨越影响边界时保持可微。

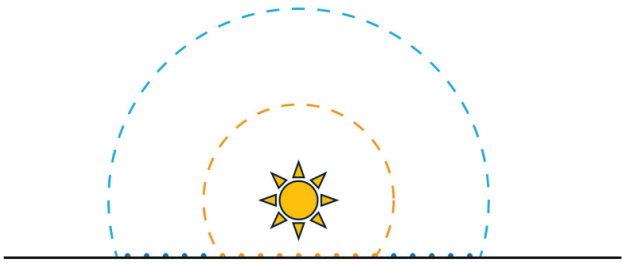

图 3 将光源作用半径扩大到两倍可保证可微性。优化时同时考虑已照亮的采样点（橙色）和未照亮的采样点（蓝色）。

对于点光源，几何惩罚项是从内部到边界距离的平滑近似，定义为

$$d_i^{\mathrm{point}}(\mathbf{x}) = \mathrm{softplus}(r_i - \|\mathbf{x} - \mathbf{s}_i\|)\,. \tag{10}$$

对于聚光灯，还须考虑光锥侧面的边界。我们取径向距离和角向边界距离的平滑最小值：

$$d_i^{\mathrm{spot}}(\mathbf{x}) = \mathrm{softmin}\big(r_i - \|\mathbf{x} - \mathbf{s}_i\|, \|\mathbf{s}_i - \mathbf{x}\|\,\sin(\theta_i^{\mathrm{o}} - \arccos(\boldsymbol{\omega}_i \cdot \mathbf{d}_i))\big)\,, \tag{11}$$

其中的平滑近似定义为

$$\mathrm{softplus}(a) = \frac{1}{\beta_p}\ln(1 + \exp(\beta_p a))\,, \tag{12}$$

$$\mathrm{softmin}(a, b) = \frac{a\exp(-\beta_m a) + b\exp(-\beta_m b)}{\exp(-\beta_m a) + \exp(-\beta_m b)}\,, \tag{13}$$

ß_p 和 ß_m 为用户设定的平滑参数。所有实验均取 ß_w = ß_p = ß_m = 1。

## 3.4 吸引损失

重叠损失会把光源从灯光密集的区域推开，能够有效消除冗余照明，也可能造成局部照明不足。单靠式（2）的第一项不足以处理这种情形。如果一个采样点落在所有现有光源的作用范围之外，该点几乎无法提供有效梯度来吸引远处的光源。因此，我们引入几何吸引损失，促使附近光源向欠照区域移动，或扩大作用范围以覆盖该区域。欠照采样点的集合定义为

$$\mathcal{S}_{\text{under-lit}} = \{\mathbf{x} \mid \hat{E}(\mathbf{x}) \le \epsilon_{\text{u}} E(\mathbf{x})\}\,, \tag{14}$$

其中 $\varepsilon_u$ 为用户控制的阈值，本文取 0.1。对每个欠照采样点 x，我们考虑一组既满足几何条件又足够接近的候选光源：

$$\mathcal{L}_{\mathbf{x},\text{attraction}} = \{l_i | (\mathbf{s}_i - \mathbf{x}) \cdot \mathbf{n} > 0 \text{ and } \|\mathbf{s}_i - \mathbf{x}\| < \eta_i\}, \tag{15}$$

第一个条件确保光源位于表面的正面一侧。自适应搜索半径由光源的有效作用半径决定：

$$\eta_i = \left(\frac{3v}{4\pi} + r_i^3\right)^{1/3} - r_i, \tag{16}$$

v 为统一的体积阈值。按照这一定义，小范围光源获得的相对搜索扩展幅度更大，因而能更积极地补偿欠照采样点。最后，我们用简单的几何惩罚定义吸引损失：

$$l^{\text{a}} = \sum_{\mathbf{x} \in \mathcal{S}_{\text{under-lit}}} \sum_{l_i \in \mathcal{L}_{\mathbf{x},\text{attraction}}} d_i(\mathbf{x})\,. \tag{17}$$

点光源和聚光灯的距离函数 $d_i(x)$ 分别见式（10）和式（12）〔译注：原文此处写作（12），聚光灯的距离函数实际列在式（11）〕。直观而言，在减少重叠的过程中，该损失会把附近光源拉向欠照区域，或扩大其有效范围，以弥补缺失的照明。

说明 重叠损失和吸引损失都针对到光源边界的距离施加惩罚，但使用的采样点集合不同。重叠损失在多盏灯共同覆盖的区域取样，减小边界距离惩罚会促使光源消除冗余覆盖。吸引损失则从所有光源边界之外的区域取样，减小距离会促使光源向欠照区域移动。

## 3.5 光源删除

对每个光源 i，我们用它对场景辐照度场贡献的总辐照度衡量其重要性：

$$\gamma_i = \sum_{\mathbf{x}_j \in \mathcal{S}_i} E_i(\mathbf{x}_j) := \gamma(\mathcal{S}_i)\,, \tag{18}$$

$S_i$ 是光源 i 照亮的采样点集合，$x_j$ 是第 j 个采样点的位置。我们进一步将 $\gamma(S_i)$ 定义为这盏灯照亮的采样点簇的重要性。优化前，若某光源的重要性 $\gamma_i$ 低于用户指定的阈值 $\varepsilon_i$，就将它剔除，因为它对整体辐照度场的影响可以忽略。

## 3.6 光源合并

光源合并用一盏有代表性的灯替换两盏灯，从而减少光源总数。

### 筛选候选光源对

为了尽量减小外观偏差，应优先合并空间影响相似的灯。我们用两盏灯对应的表面采样点集合之间的倒角距离（Chamfer distance）衡量相似性。设 $S_1$、$S_2$ 分别为两灯照亮的点集，距离定义为

$$d(\mathcal{S}_1,\mathcal{S}_2)=\frac{1}{|\mathcal{S}_1|}\sum_{\mathbf{x}\in\mathcal{S}_1}\min_{\mathbf{p}\in\mathcal{S}_2}\|\mathbf{p}-\mathbf{x}\|^2+\frac{1}{|\mathcal{S}_2|}\sum_{\mathbf{x}\in\mathcal{S}_2}\min_{\mathbf{p}\in\mathcal{S}_1}\|\mathbf{p}-\mathbf{x}\|^2$$

光源 i 与 j 的合并代价定义为

$$M(i,j)=\gamma_i\,\gamma_j\,d(\mathcal{S}_i,\mathcal{S}_j)\,, \tag{19}$$

其中 $\gamma_i$ 和 $\gamma_j$ 表示两盏灯各自的相对重要性。

### 初始化合并后的光源

按上述分数取最低的 10% 候选光源对。只有当新光源能降低重叠损失，且超过 80% 的采样点以正面朝向新光源时，才合并该对光源。用原光源的重要性作权重，对颜色、位置和衰减平滑参数取加权平均，作为新光源的初值。再根据加权后的位置确定作用半径和聚光锥，使新光源完全覆盖原来的两个受光点集。若所需聚光角超过预设阈值 A_point，就改用点光源，否则保留为聚光灯。这样既能为合并后的光源提供可靠初值，也为后续优化留出微调空间。聚光角的定义域为 0° 至 180° ，本文取 A_point = 130° 。实际制作中，灯光美术很少使用超过 130° 的聚光角，我们也沿用这一经验。

## 3.7 光源添加

光源删除、合并和参数优化都可能导致原先照亮的地方失去足够照明。连续优化可以缓解轻度欠照，但如果附近没有光源，就无法恢复缺失的照明。因此，我们也会在优化过程中添加新光源。

先按第 3.4 节找出欠照采样点，再用 DBSCAN［Ester 等 1996］按采样点位置的欧氏距离聚类。如果一个簇的累计重要性超过第 3.5 节中的阈值 $\varepsilon_l$，就生成一盏新灯来补足照明。新灯根据簇的统计量初始化：用簇内平均目标辐照度和平均法线设置初始颜色、强度与方向。从采样点的平均位置出发，沿平均法线搜索，使簇内全部采样点的辐照度差异最小，以此确定光源位置。最后按照合并时的同一判据选择点光源或聚光灯。

### 3.8 昼夜时变照明

在实际的实时渲染管线中，照明配置常随时间变化。例如，昼夜时变照明（ToD）的光照强度、颜色及启用状态会随时间持续变化。引入这一时间变化，会让优化问题更复杂。在生产系统中，每个 ToD 光源 i 通常由一组稀疏关键帧 $T_i = \{t_1, \cdots, t_K_i\}$ 指定，帧间参数采用分段线性插值。为支持 ToD 照明，我们对所有光源的关键帧集合取并集，得到全局时间集合 T。随后在式（2）中显式引入时间维度，并在这一组关键帧上进行数值积分：

$$\arg\min_{\{\mathbf{c}_i, I_i, \mathbf{s}_i, \mathbf{d}_i, \dots\}} \sum_{k=1}^{|\mathcal{T}|} w_k \left[ \Delta s \sum_{\mathbf{x} \in \mathcal{S}_{\text{all}}} \left( \hat{E}_k(\mathbf{x}) - E_k(\mathbf{x}) \right)^2 + a^{\text{o}} l_k^{\text{o}} + a^{\text{a}} l_k^{\text{a}} \right], \quad (20)$$

其中 $w_k$ 是非负求积权重，所有权重之和为 1。由于关键帧在时间上并不均匀，我们用非等距梯形法则计算权重。该形式与常规游戏引擎管线相符：引擎在离散关键帧上求取 ToD 光照参数，并在帧间连续插值。

## 4 实验结果

LightOpt 在预处理阶段优化光源，可直接接入现有的分块渲染管线，无需更改分块渲染算法。我们使用 Unity［Unity 2023］在五个真实游戏场景中评估本方法，延迟着色管线［Olsson 与 Assarsson 2011］采用 Tiled-Z 剔除。所有结果均在 Intel i9-12900KF 与 NVIDIA GeForce RTX 3090 上以 1920 × 1080 分辨率实时渲染。

### 测试场景

我们在五个真实游戏场景中进行对比实验、性能评测和消融研究，涵盖游戏常见的多种室内外照明环境，包括酒馆（Tavern）、山谷（Valley）、广场（Square，使用 ToD 灯光）、洞穴（Cave）和街道（Street）。场景由专业灯光美术布置，光源数量从 80 盏到 300 多盏不等。每个场景的训练时间约为 10 分钟。实验采用的均为开放世界环境，尺寸约 16 km × 16 km。我们按场景分区，将光源分成多个空间批次进行优化。

### 评价指标

以均方根误差（RMSE）衡量照明差异。我们还引入平均光照重叠数（average light overdraw，ALO），用屏幕上每个像素对应点的 $|L_x|$（见第 3.3 节）衡量逐像素光照重叠。我们用 Nsight Graphics 测量延迟着色阶段的 GPU 执行时间来评价性能。每个场景选取至多八个有代表性的相机位姿，分别报告这些视角下 RMSE、ALO 和延迟着色耗时的平均值与最大值。

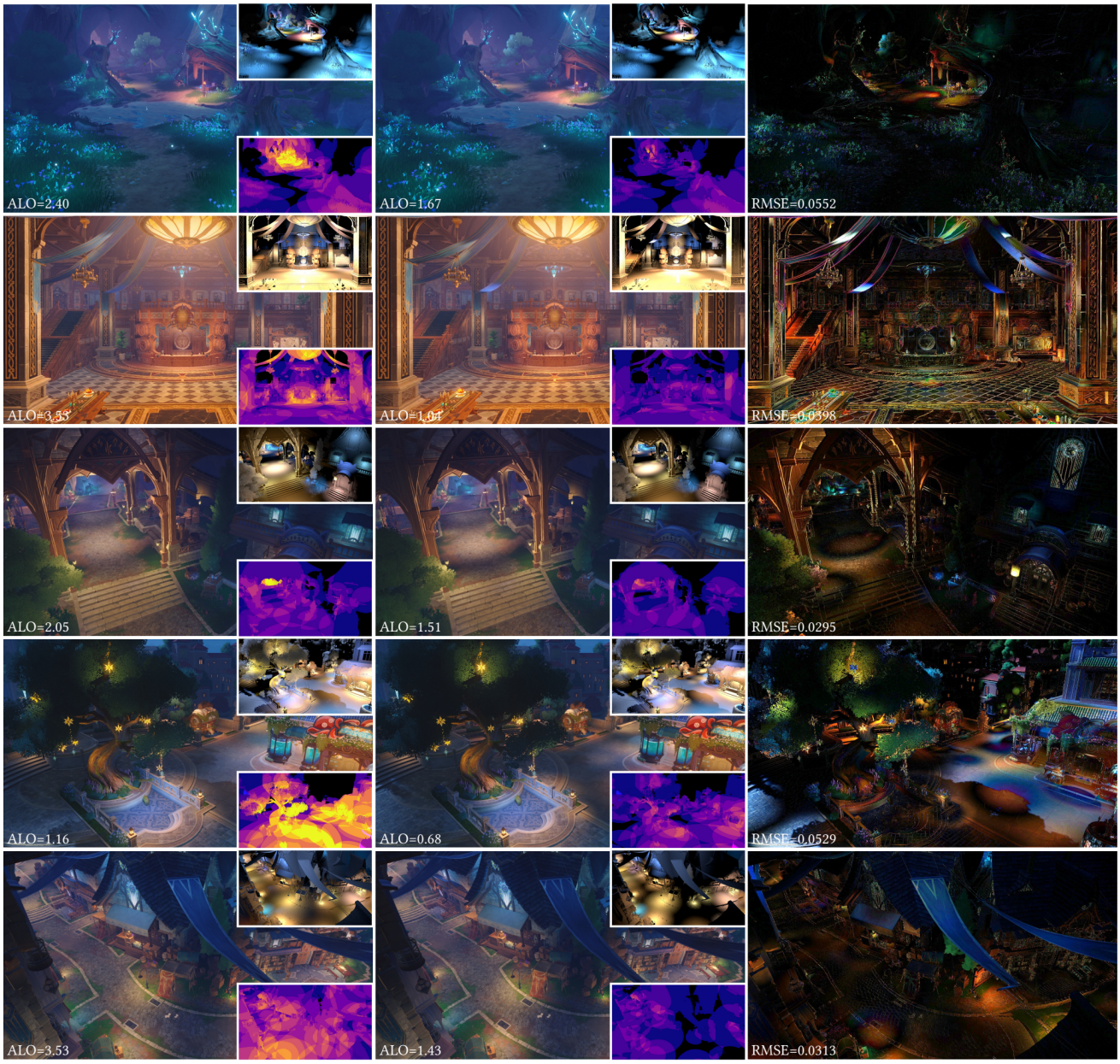


图 4 山谷、酒馆、洞穴、广场及街道的渲染质量对比。各列从左至右依次为原始光照、本方法优化后的光照，以及二者逐像素绝对误差放大 5 倍后的图像。右上角的小图采用白色材质（移除反照率）突出光照差异；右下角的小图显示逐像素光照重叠。

## 渲染质量

图 4 对比了原始和优化后的照明。除了采用完整材质的渲染图，我们还给出白模渲染图，以便观察两种配置产生的辐照度场。即使大幅精简光源，也没有出现大面积欠照或过照。这说明优化后的光源参数能稳定近似原始外观，仅在局部出现轻微差异。绝对差值图显示，LightOpt 较好地保持了外观；光照重叠图则显示逐像素重叠明显减少，相应的预期着色开销也随之下降，ALO 最多下降 50%。

## 消融实验

为评估各组成部分的贡献，我们逐一关闭单个模块，其他设置保持一致。所有场景中固定吸引损失权重 $a^a = 0.2$、采样半径 $r_0 = 0.25$，默认重叠损失权重 $a^o = 0.1$、吸引体积 $v = 15$、重要性阈值 $\varepsilon_i = 4$。为了支持跨平台部署，设最大允许重叠阈值 n_max = 3。

## 重叠损失

重叠损失惩罚过度重叠，从而减少影响每个像素的光源数，降低逐像素照明开销。图 5 展示了改变重叠权重 $a^o$ 后质量和效率之间的折中。增大权重更能消除重叠，但也允许更大的照明偏差，因此 ALO 降低，RMSE 却会上升。

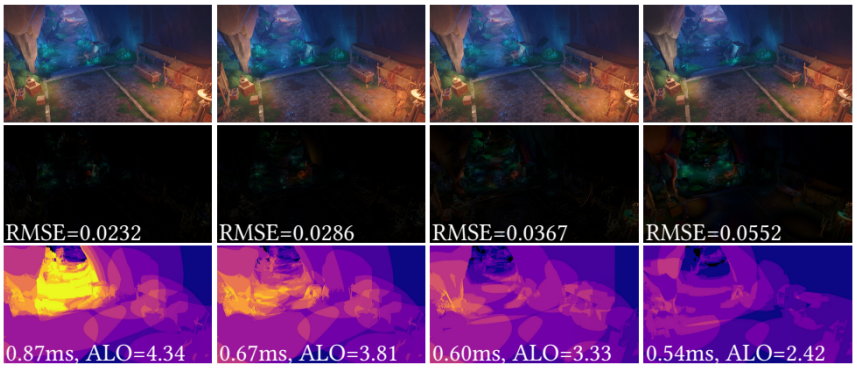


图 5 重叠损失消融。自左至右，$a^o$ 依次为 0、0.001、0.1、5。自上至下依次为最终渲染、逐像素误差、逐像素光照重叠。评测平台为 RTX 3090。

## 吸引损失

吸引损失促使附近光源移向欠照区域。为了单独检验其作用，我们将吸引体积设为 v = 0，以移除吸引区域，阻止光源补偿缺失的照明。如图 6 所示，移除吸引损失会造成明显欠照，明暗交界生硬。加入吸引损失使光源能探测附近的欠照区域，从而缓解问题。但吸引体积过大，会使光源过度扩张，把光分配到本应保持黑暗的区域，损失精细照明细节。

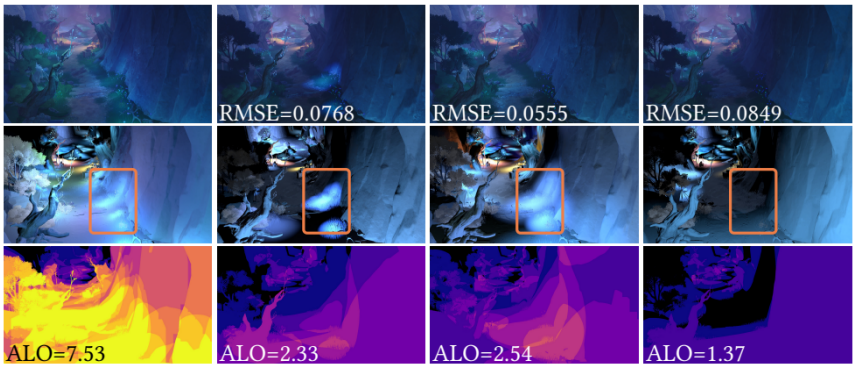


图 6 吸引体积 v 的消融。从左到右为输入、v = 0、v = 15、v = 100；从上到下为最终渲染、白模和逐像素光照重叠。橙框显示，去掉吸引损失（v = 0）后，新加的灯可能在优化中被推离高重叠区域，导致局部欠照。

## 动态调整光源

在训练中动态删除、合并和添加光源，使优化器能够调整光源数量，改善最终照明。如图 7 所示，关闭动态调整后，标记区域的光照重叠水平相近，实际照明却明显不同，关闭该机制的版本容易留下欠照。把动态控制与吸引损失结合，可以消除大部分欠照伪影。

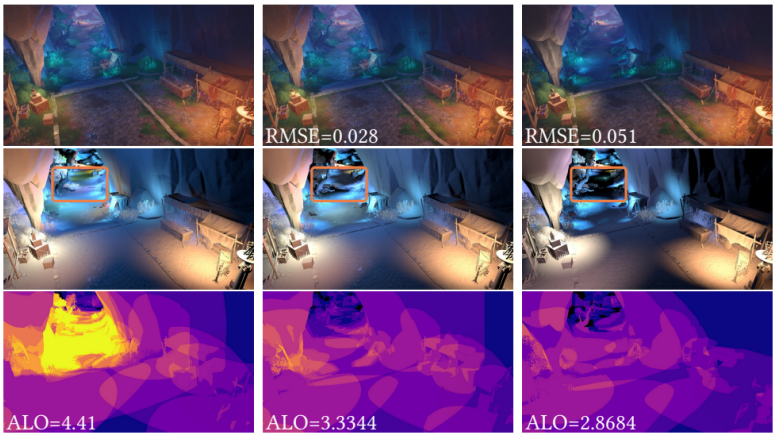


图 7 动态光源调整的消融。各列依次为输入、本方法、关闭动态调整的本方法。各行依次为最终渲染、白模和逐像素光照重叠。橙框显示，动态调整减少了欠照伪影，同时维持了适当的重叠水平。

我们还测试了删除和添加光源时采用的不同重要性阈值 $\varepsilon_i$。如图 8 标出的区域所示，较大的阈值会过度删除作用距离短、重要性较低的灯，导致局部欠照和细小渲染细节丢失。此外，阈值过大会使候选灯在合并之前就被删除，后续只能合并剩余光源，合并出的灯往往作用范围过大，反而可能提高 ALO。

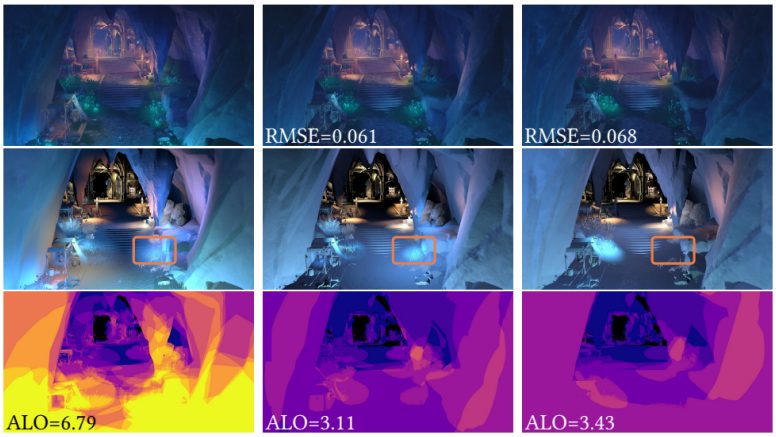


图 8 重要性阈值 $\varepsilon_i$ 的消融。各列依次为输入、$\varepsilon_i = 4$、$\varepsilon_i = 8$；各行依次为最终渲染、白模渲染和逐像素光照重叠。

吸引损失与动态补灯共同处理欠照采样点。表 1 给出五个场景的平均与最大 RMSE。当附近已有能合理覆盖缺光区域的灯时，吸引损失就能发挥作用。如果附近没有合适的光源，就必须通过新加光源引入新的自由度。

表 1 吸引损失与光源添加的消融对比。每格依次给出照明差异的平均／最大 RMSE。

| 方法 | 山谷 | 酒馆 | 广场 | 洞穴 | 街道 |
|---|---|---|---|---|---|
| 无吸引损失 | 0.066 / 0.106 | 0.074 / 0.089 | 0.098 / 0.110 | 0.078 / 0.103 | 0.080 / 0.087 |
| 无光源添加 | 0.053 / 0.089 | 0.067 / 0.082 | 0.096 / 0.109 | 0.039 / 0.055 | 0.066 / 0.073 |
| 两者均启用 | 0.045 / 0.068 | 0.041 / 0.061 | 0.043 / 0.053 | 0.037 / 0.049 | 0.033 / 0.034 |

## 与业界常用方法及手工精简的对比

业界一种常见做法是在运行时收集视锥内的全部光源，根据它们在相机焦点处的贡献排序，再按固定预算截断，将选中的光源提交给渲染管线。因此，结果与视角有关。在我们的对比设置中，运行时每个视锥严格限定最多 32 盏灯。如图 9 所示，业界基线方法还需要相机位姿作为输入，倾向于保留近处灯光、舍弃远处灯光，带来的加速有限，却使远处照明细节明显退化。美术人员手工制作的版本为满足低端设备的实时要求，大幅削减照明，但 RMSE 明显更高。

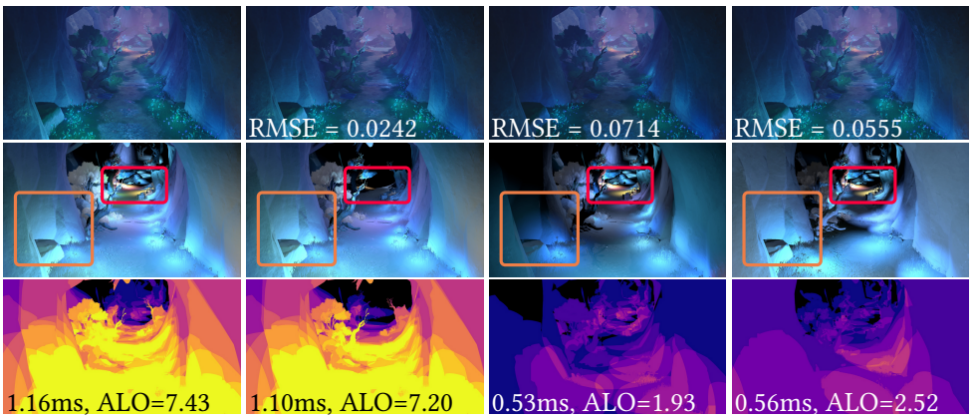


图 9 不同方法之间的对比。各列依次为输入、业界方法、手工精简、本方法。各行依次为最终渲染、白模渲染、逐像素光照重叠。

## 性能

如表 2 所示，在所有测试场景中，本方法均降低了 ALO，并加快了延迟着色。业界方法根据相机位姿剔除不重要的光源，却丢失了更多细节，加速幅度也更小。在这些场景中，LightOpt 的平均加速比（输入耗时／本方法耗时）分别为山谷 1.67 倍、酒馆 1.37 倍、广场 1.48 倍、洞穴 1.44 倍、街道 1.52 倍。在复杂光照设置下，最高达到 2.07 倍（图 9）。

表 2 原始配置、业界方案、美术手工精简及本方法的定量比较。各组指标均按平均值／最大值列出。* 业界方案在每个视口最多保留 32 盏灯，保留的灯随视口而变。† 使用 ToD 照明。‡ 由美术人员手工制作。

| 场景 | 方案 | 灯数 | ALO | 着色耗时（ms） | RMSE |
|---|---|---|---|---|---|
| 山谷 | 输入 | 121 | 5.78 / 7.86 | 0.95 / 1.16 | — |
| | 业界 | 32* | 4.46 / 7.05 | 0.75 / 1.10 | 0.055 / 0.076 |
| | 手工‡ | 83 | 2.12 / 2.60 | 0.54 / 0.59 | 0.073 / 0.107 |
| | 本方法 | 57 | 2.54 / 3.76 | 0.57 / 0.63 | 0.046 / 0.068 |
| 酒馆 | 输入 | 210 | 3.35 / 4.06 | 0.71 / 0.80 | — |
| | 业界 | 32* | 2.51 / 3.09 | 0.58 / 0.60 | 0.071 / 0.104 |
| | 本方法 | 126 | 1.72 / 1.97 | 0.52 / 0.55 | 0.041 / 0.061 |
| 广场† | 输入 | 314 | 3.28 / 5.43 | 0.83 / 1.16 | — |
| | 业界 | 32* | 2.00 / 4.17 | 0.60 / 0.93 | 0.073 / 0.086 |
| | 本方法 | 249 | 1.57 / 2.13 | 0.56 / 0.68 | 0.043 / 0.053 |
| 洞穴 | 输入 | 80 | 3.43 / 6.13 | 0.71 / 1.05 | — |
| | 业界 | 32* | 2.73 / 5.35 | 0.60 / 0.88 | 0.041 / 0.056 |
| | 本方法 | 64 | 2.05 / 3.30 | 0.49 / 0.60 | 0.037 / 0.049 |
| 街道 | 输入 | 135 | 3.83 / 4.82 | 0.67 / 0.72 | — |
| | 业界 | 32* | 3.24 / 3.79 | 0.55 / 0.62 | 0.037 / 0.045 |
| | 本方法 | 95 | 1.68 / 2.23 | 0.45 / 0.49 | 0.033 / 0.034 |

## 昼夜时变照明

我们在启用 ToD 灯光的广场场景中评估 LightOpt。图 10 显示，该方法可以有效优化随时间变化的照明。广场的相机漫游视频也展示了不同时刻之间的过渡。

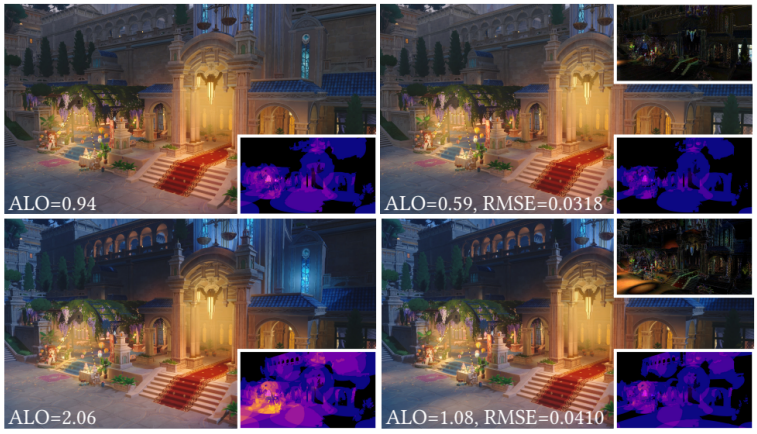


图 10 启用昼夜时变照明的广场场景。左列为输入，右列为本方法的优化结果。右下角显示逐像素光照重叠，右上角显示放大 5 倍的逐像素绝对误差。

## 跨平台性能

为检验 LightOpt 在不同硬件平台上的效果，除前述高端 GPU 之外，我们还在中端和低端设备上进行了测试。中端平台使用 Intel i9-9900 CPU 和 NVIDIA GeForce RTX 2080 GPU，低端平台使用 Intel UHD Graphics 630 集成 GPU。我们用 NVIDIA Nsight Graphics 测量高端和中端平台延迟着色阶段的 GPU 时间。由于 Nsight Graphics 不支持集成 GPU，低端平台用 RenderDoc 估算延迟着色耗时。虽然使用不同的分析工具，两种测量仍能一致反映逐像素光照计算的相对开销。

表 3 给出了各场景平均和最大延迟着色耗时。LightOpt 持续降低着色开销，在高、中、低端设备上的相对加速幅度相近。值得注意的是，着色耗时的绝对降幅在低端硬件上最大，这体现了它在性能受限平台上的实际价值。例如，在山谷场景中，集成 GPU 的平均延迟着色时间最多缩短了 39.7 ms。

表 3 跨平台延迟着色性能。各格均为平均值／最大值。GPU 时间以 ms 计，加速比为输入耗时除以本方法耗时。高、中、低端平台分别为 RTX 3090、RTX 2080 和 Intel UHD Graphics 630。† 使用 ToD 照明。

| 场景 | 平台 | 输入（ms） | 本方法（ms） | 加速比（倍） |
|---|---|---|---|---|
| 山谷 | 高端 | 0.95 / 1.16 | 0.57 / 0.63 | 1.67 / 1.84 |
| | 中端 | 3.00 / 3.87 | 1.76 / 1.96 | 1.70 / 1.97 |
| | 低端 | 117.22 / 146.40 | 77.50 / 91.79 | 1.51 / 1.59 |
| 酒馆 | 高端 | 0.71 / 0.80 | 0.52 / 0.55 | 1.37 / 1.45 |
| | 中端 | 2.21 / 2.39 | 1.64 / 1.69 | 1.35 / 1.41 |
| | 低端 | 110.89 / 113.41 | 84.55 / 87.84 | 1.31 / 1.29 |
| 广场† | 高端 | 0.83 / 1.16 | 0.56 / 0.68 | 1.48 / 1.71 |
| | 中端 | 2.01 / 2.05 | 1.55 / 1.57 | 1.30 / 1.31 |
| | 低端 | 84.73 / 93.30 | 69.90 / 72.27 | 1.21 / 1.29 |
| 洞穴 | 高端 | 0.71 / 1.05 | 0.49 / 0.60 | 1.45 / 1.75 |
| | 中端 | 2.17 / 3.15 | 1.62 / 2.01 | 1.34 / 1.57 |
| | 低端 | 91.20 / 127.05 | 75.19 / 92.11 | 1.21 / 1.38 |
| 街道 | 高端 | 0.67 / 0.72 | 0.45 / 0.49 | 1.49 / 1.47 |
| | 中端 | 2.05 / 2.16 | 1.42 / 1.49 | 1.44 / 1.45 |
| | 低端 | 98.96 / 103.58 | 70.41 / 73.15 | 1.41 / 1.42 |

### 困难情形

含有植被、薄片几何或光照高度重叠区域的场景，往往很难准确保留辐照度。图 11 所示场景包含植被和强烈的光照重叠，优化后出现了欠照。虽然本方法无法在这些困难情形下完美保留辐照度，整个场景的照明结果仍然基本合理。

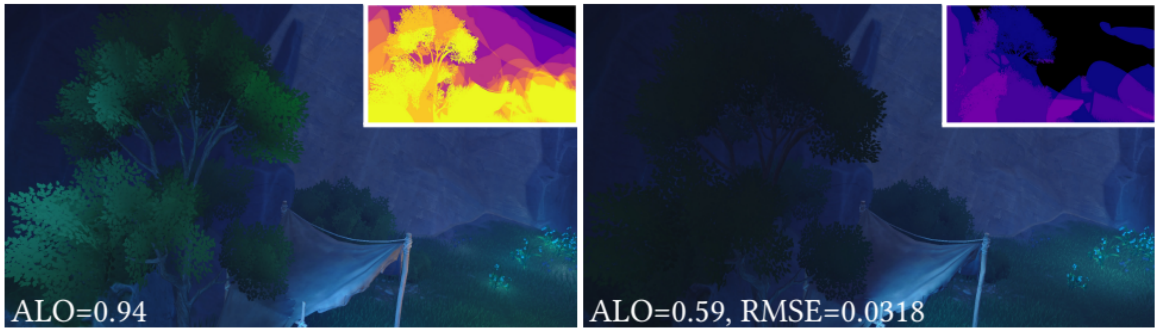


图 11 植被与强烈光照重叠同时存在的困难情形。左图为输入，右图为优化后的光照。右上角的小图显示逐像素光照重叠。

## 5 结论

本文提出一种可微光照优化流程，在保持实时游戏场景视觉保真度的同时，自动减少活跃光源。通过联合优化灯光参数并动态重组光源集合，我们在无需人工干预的情况下降低了光照开销。两个损失函数在减少冗余和保持覆盖之间取得平衡。对昼夜时变照明的扩展，使方法在动态光照条件下也能产生一致的结果。该方法减轻了美术人员的工作量，改善了跨平台扩展能力，为现代实时渲染管线中的高效照明提供了实用方案。

### 局限性

我们把保留辐照度场作为优化目标，以近似最终着色图像。尽管这种近似在实践中表现良好，通常可以保持延迟着色下的视觉效果，但它没有显式建模镜面高光等与视角有关的效果。因此，在高光泽材质区域或照明急剧变化的位置，可能出现细微偏差。引入基于感知或考虑 BRDF 的目标，有望提高这些情形下的精度。

此外，动态调整光源包含删除、合并和插入等离散操作。让这些操作完全可微，是值得探索的方向。目前我们还假设待优化光源的可见性恒为 1，因为在实际游戏管线中，受性能限制，只有少数光源投射阴影。扩展框架以优化投射阴影的光源，也是一个重要的后续方向，这要求在可微优化中处理不连续的可见性。

最后，当前框架尚未显式支持面光源。面光源需要比点光源和聚光灯更复杂的可微处理，但这项扩展不存在根本障碍。在许多实时游戏中，面光源实际上较少使用。环境照明也在本文的范围之外，因为它通常由灯光美术制作，或作为全局照明的一部分预先烘焙。

### 致谢

本工作获得国家自然科学基金项目（编号 62220106003、62495060）及清华大学 – 腾讯互联网创新技术联合实验室研究经费的部分资助。

# 参考文献


### 01 Frostbite 中的并行图形技术：现状与未来

Johan Andersson. 2009. Parallel graphics in frostbite-current & future. In ACM SIGGRAPH 2009 Courses. Association for Computing Machinery, New York, NY, USA.

### 02 复杂灯具的小波表示与采样

A. Atanasov and V. Koylazov. 2025. Wavelet Representation and Sampling of Complex Luminaires. Computer Graphics Forum 44, 4, e70163.

### 03 用于实时光线追踪动态直接照明的时空蓄水池重采样

Benedikt Bitterli, Chris Wyman, Matt Pharr, Peter Shirley, Aaron Lefohn, and Wojciech Jarosz. 2020. Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting. ACM Transactions on Graphics (TOG) 39, 4, Article 148 (Aug. 2020), 17 pages.

### 04 利用自适应树分裂对大量光源进行重要性采样

Alejandro Conty Estevez and Christopher Kulla. 2018. Importance sampling of many lights with adaptive tree splitting. Proceedings of the ACM on Computer Graphics and Interactive Techniques 1, 2 (2018), 1–17.

### 05 基于多光源方法的可扩展真实感渲染

Carsten Dachsbacher, Jaroslav Křivánek, Miloš Hašan, Adam Arbree, Bruce Walter, and Jan Novák. 2014. Scalable Realistic Rendering with Many-Light Methods. Computer Graphics Forum 33, 1 (Feb. 2014), 88–104.

### 06 通过强化学习学习光传输

Ken Dahm and Alexander Keller. 2017. Learning light transport the reinforced way. In ACM SIGGRAPH 2017 Talks (Los Angeles, California) (SIGGRAPH ’17). Association for Computing Machinery, New York, NY, USA, Article 73, 2 pages.

### 07 用于实时直接照明的马尔可夫链混合模型

Addis Dittebrandt, Vincent Schubler, Johannes Hanika, Sebastian Herholz, and Carsten Dachsbacher. 2023. Markov Chain Mixture Models for Real-Time Direct Illumination. Computer Graphics Forum 42, 4 (2023), e14881.

### 08 改进分块与分簇渲染的剔除方法

Michal Drobot. 2017. Improved Culling for Tiled and Clustered Rendering. In ACM SIGGRAPH 2017 Courses. Association for Computing Machinery, NY, USA.

### 09 在含噪大型空间数据库中发现簇的密度算法

Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In ACM KDD’96: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, Vol. 96. 226–231.

# 参考文献（续）


### 10 用于镜面路径链重要性采样的流形路径引导

Zhimin Fan, Pengpei Hong, Jie Guo, Changqing Zou, Yanwen Guo, and Ling-Qi Yan. 2023. Manifold path guiding for importance sampling specular chains. ACM Transactions on Graphics (TOG) 42, 6 (2023), 1–14.

### 11 用于路径引导的多重重要性重加权

Zhimin Fan, Yiming Wang, Chenxi Zhou, Ling-Qi Yan, Yanwen Guo, and Jie Guo. 2025. Multiple Importance Reweighting for Path Guiding. ACM Transactions on Graphics (TOG) 44, 4 (2025), 1–11.

### 12 用蒙特卡罗渲染与降噪从图像中分解形状、光照和材质

Jon Hasselgren, Nikolai Hofmann, and Jacob Munkberg. 2022. Shape, light, and material decomposition from images using monte carlo rendering and denoising. Advances in Neural Information Processing Systems 35 (2022), 22856–22869.

### 13 外观驱动的三维模型自动简化

Jon Hasselgren, Jacob Munkberg, Jaakko Lehtinen, Miika Aittala, and Samuli Laine. 2021. Appearance-Driven Automatic 3D Model Simplification. EGSR (DL) 29 (2021), 85–97.

### 14 用于多光源问题的矩阵行列采样

Miloš Hašan, Fabio Pellacini, and Kavita Bala. 2007. Matrix row-column sampling for the many-light problem. ACM Transactions on Graphics (TOG) 26, 3 (2007), 26–es.

### 15 用于渲染多光源动画的张量聚类

Miloš Hašan, Edgar Velázquez-Armendariz, Fabio Pellacini, and Kavita Bala. 2008. Tensor clustering for rendering many-light animations. In Proceedings of the Nineteenth Eurographics Conference on Rendering (Sarajevo, Bosnia and Herzegovina) (EGSR ’08). Eurographics Association, Goslar, DEU, 1105–1114.

### 16 光传输路径引导中的乘积重要性采样

Sebastian Herholz, Oskar Elek, Jiří Vorba, Hendrik Lensch, and Jaroslav Křivánek. 2016. Product Importance Sampling for Light Transport Path Guiding. Computer Graphics Forum 35, 4 (2016), 67–77.

### 17 利用半球粒子足迹进行重要性采样

Heinrich Hey and Werner Purgathofer. 2002. Importance sampling with hemispherical particle footprints. In Proceedings of the 18th Spring Conference on Computer Graphics (SCCG ’02). Association for Computing Machinery, New York, NY, USA, 107–114.

### 18 利用光子图进行重要性驱动的路径追踪

Henrik Wann Jensen. 1995. Importance Driven Path Tracing using the Photon Map. In Rendering Techniques ’95, Patrick M. Hanrahan and Werner Purgathofer (Eds.). Springer Vienna, Vienna, 326–335.

# 参考文献（续）


### 19 UV 联合优化与纹理烘焙

Julian Knodt, Zherong Pan, Kui Wu, and Xifeng Gao. 2023. Joint UV Optimization and Texture Baking. ACM Transactions on Graphics (TOG) 43, 1 (2023), 20 pages.

### 20 用于降低蒙特卡罗光线追踪方差的五维树

Eric P. Lafortune and Yves D. Willems. 1995. A 5D Tree to Reduce the Variance of Monte Carlo Ray Tracing. In Rendering Techniques ’95, Patrick M. Hanrahan and Werner Purgathofer (Eds.). Springer Vienna, Vienna, 11–20.

### 21 Frostbite 向基于物理的渲染 3.0 迁移

Sebastien Lagarde and Charles De Rousiers. 2014. Moving frostbite to physically based rendering 3.0. SIGGRAPH Course 3 (2014).

### 22 面向当前与未来渲染管线的延迟渲染

Andrew Lauritzen. 2010. Deferred rendering for current and future rendering pipelines. In ACM SIGGRAPH 2010 Courses. Association for Computing Machinery, New York, NY, USA, 1–34.

### 23 由代表性镜面路径引导的无偏焦散渲染

He Li, Beibei Wang, Changhe Tu, Kun Xu, Nicolas Holzschuch, and Ling-Qi Yan. 2022. Unbiased Caustics Rendering Guided by Representative Specular Paths. In SIGGRAPH Asia 2022 Conference Papers. Association for Computing Machinery, New York, NY, USA, Article 41, 8 pages.

### 24 通过边缘采样实现可微蒙特卡罗光线追踪

Tzu-Mao Li, Miika Aittala, Frédo Durand, and Jaakko Lehtinen. 2018. Differentiable monte carlo ray tracing through edge sampling. ACM Transactions on Graphics (TOG) 37, 6 (2018), 1–11.

### 25 梯度域渲染的自适应采样与重建

Yuzhi Liang, Tao Liu, Yuchi Huo, Rui Wang, and Hujun Bao. 2024. Adaptive sampling and reconstruction for gradient-domain rendering. Computational Visual Media 10, 5 (2024), 885–902.

### 26 实时随机 Lightcuts

Daqi Lin and Cem Yuksel. 2020. Real-time stochastic lightcuts. Proceedings of the ACM on Computer Graphics and Interactive Techniques 3, 1 (2020), 1–18.

### 27 机器学习峰会：《王者荣耀》可扩展资产管线中的可微渲染

Fei Ling and Frei Zhang. 2023. Machine Learning Summit: Differentiable Rendering for Scalable Asset Pipeline in ’Honor of Kings’. GDC Vault, Game Developers Conference 2023, Tools Summit presentation. Tencent Technology Inc..

# 参考文献（续）


### 28 用于照明设计优化的视角无关伴随光线追踪

Lukas Lipp, David Hahn, Pierre Ecormier-Nocca, Florian Rist, and Michael Wimmer. 2024. View-Independent Adjoint Light Tracing for Lighting Design Optimization. ACM Trans. Graph. 43, 3, Article 35 (May 2024), 16 pages.

### 29 基于包围体素采样的实时路径引导

Haolin Lu, Wesley Chang, Trevor Hedstrom, and Tzu-Mao Li. 2024. Real-Time Path Guiding Using Bounding Voxel Sampling. ACM Transactions on Graphics (TOG) 43, 4, Article 125 (July 2024), 14 pages.

### 30 用于实时光线追踪的动态多光源采样

P. Moreau, M. Pharr, and P. Clarberg. 2022. Dynamic many-light sampling for real-time ray tracing. In Proceedings of the Conference on High-Performance Graphics (Strasbourg, France) (HPG ’ 19). Eurographics Association, Goslar, DEU, 21–26.

### 31 用于高效光传输模拟的实用路径引导

Thomas Müller, Markus Gross, and Jan Novák. 2017. Practical Path Guiding for Efficient Light-Transport Simulation. Computer Graphics Forum 36, 4 (July 2017), 91–100.

### 32 从图像提取三角网格三维模型、材质和光照

Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. 2022. Extracting triangular 3d models, materials, and lighting from images. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition . 8280–8290.

### 33 MegaLights：虚幻引擎 5 中的随机直接照明

Krzysztof Narkowicz and Tiago Costa. 2025. MegaLights: Stochastic Direct Lighting in Unreal Engine 5. SIGGRAPH 2025 Course: Advances in Real-Time Rendering in Games. Course presentation.

### 34 Mitsuba 2：可重定向的正向与逆向渲染器

Merlin Nimier-David, Delio Vicini, Tizian Zeltner, and Wenzel Jakob. 2019. Mitsuba 2: A retargetable forward and inverse renderer. ACM Transactions on Graphics (ToG) 38, 6 (2019), 1–17.

### 35 分块着色

Ola Olsson and Ulf Assarsson. 2011. Tiled shading. Journal of Graphics, GPU, and Game Tools 15, 4 (2011), 235–251.

### 36 分簇延迟着色与前向着色

Ola Olsson, Markus Billeter, and Ulf Assarsson. 2012. Clustered deferred and forward shading. In Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics. Eurographics Association, Goslar, DEU, 87–96.

# 参考文献（续）


### 37 LightSlice：用于多光源问题的矩阵切片采样

Jiawei Ou and Fabio Pellacini. 2011. LightSlice: matrix slice sampling for the many-lights problem. ACM Transactions on Graphics (TOG) 30, 6 (2011), 179.

### 38 实用分簇着色

Emil Persson. 2013. Practical clustered shading. SIGGRAPH Course: Advances in Real- Time Rendering in Games (2013).

### 39 Unity Technologies

Unity. 2023. Unity Technologies. https://unity.com/

### 40 复杂灯具：光照与外观渲染

Edgar Velázquez-Armendáriz, Zhao Dong, Bruce Walter, and Donald P Greenberg. 2015. Complex luminaires: Illumination and appearance rendering. ACM Transactions on Graphics (TOG) 34, 3 (2015), 1–15.

### 41 实际制作中的路径引导

Jiří Vorba, Johannes Hanika, Sebastian Herholz, Thomas Müller, Jaroslav Křivánek, and Alexander Keller. 2019. Path guiding in production. In ACM SIGGRAPH 2019 Courses (Los Angeles, California) (SIGGRAPH ’19). Association for Computing Machinery, New York, NY, USA, Article 18, 77 pages.

### 42 面向光传输模拟的参数混合模型在线学习

Jiří Vorba, Ondřej Karlík, Martin Šik, Tobias Ritschel, and Jaroslav Křivánek. 2014. On-line learning of parametric mixture models for light transport simulation. ACM Transactions on Graphics (TOG) 33, 4 (2014), 1–11.

### 43 多维 Lightcuts

Bruce Walter, Adam Arbree, Kavita Bala, and Donald P. Greenberg. 2006. Multidimensional lightcuts. ACM Transactions on Graphics (TOG) 25, 3 (July 2006), 1081–1088.

### 44 Lightcuts：一种可扩展的光照计算方法

Bruce Walter, Sebastian Fernandez, Adam Arbree, Kavita Bala, Michael Donikian, and Donald P. Greenberg. 2005. Lightcuts: a scalable approach to illumination. ACM Transactions on Graphics (TOG) 24, 3 (July 2005), 1098–1107.

### 45 双向 Lightcuts

Bruce Walter, Pramook Khungurn, and Kavita Bala. 2012. Bidirectional lightcuts. ACM Transactions on Graphics (TOG) 31, 4 (2012), 1–11.

# 参考文献（续）


### 46 面向实际制作重构时空重采样架构

Chris Wyman and Alexey Panteleev. 2022. Rearchitecting spatiotemporal resampling for production. In Proceedings of the Conference on High-Performance Graphics (HPG ’ 21). Eurographics Association, Goslar, DEU, 23–41.

### 47 基于可微渲染的动态海洋逆向建模

Xueguang Xie, Yang Gao, Fei Hou, Aimin Hao, and Hong Qin. 2024. Dynamic ocean inverse modeling based on differentiable rendering. Computational Visual Media 10, 2 (2024), 279–294.

### 48 对方差求导以实现方差感知的逆向渲染

Kai Yan, Vincent Pegoraro, Marc Droske, Jiří Vorba, and Shuang Zhao. 2024. Differentiating Variance for Variance-Aware Inverse Rendering. In SIGGRAPH Asia 2024 Conference Papers . Association for Computing Machinery, New York, NY, USA, Article 117, 10 pages.

### 49 通过样本消除生成泊松盘采样点集

Cem Yuksel. 2015. Sample Elimination for Generating Poisson Disk Sample Sets. Computer Graphics Forum 34, 2 (2015), 25–32.

### 50 随机 Lightcuts

Cem Yuksel. 2019. Stochastic lightcuts. In High-Performance Graphics (HPG 2019) . The Eurographics Association, Strasbourg, France, 27–32.

### 51 ReSTIR PG：利用时空重采样路径进行路径引导

Zheng Zeng, Markus Kettunen, Chris Wyman, Lifan Wu, Ravi Ramamoorthi, Ling-Qi Yan, and Daqi Lin. 2025. ReSTIR PG: Path Guiding with Spatiotemporally Resampled Paths. In Proceedings of the SIGGRAPH Asia 2025 Conference Papers . Association for Computing Machinery, New York, NY, USA, Article 12, 11 pages.

### 52 用可微渲染为柔顺毛发自动提取发片

Zhongtian Zheng, Tao Huang, Haozhe Su, Xueqi Ma, Yuefan Shen, Tongtong Wang, Yin Yang, Xifeng Gao, Zherong Pan, and Kui Wu. 2025. Auto Hair Card Extraction for Smooth Hair with Differentiable Rendering. ACM Transactions on Graphics (TOG) 44, 6 (2025), 1–13.

### 53 神经复杂灯具：表示与渲染

Junqiu Zhu, Yaoyi Bai, Zilin Xu, Steve Bako, Edgar Velázquez-Armendáriz, Lu Wang, Pradeep Sen, Milos Hasan, and Ling-Qi Yan. 2021. Neural complex luminaires: representation and rendering. ACM Trans. Graph. 40, 4 (2021), 57–1.